SMART TECHNOLOGIES / PS1 + PS3

AEGIS / Start Here

Beginner setup and first successful run

Four teammates. One shared GPU. Protected PPT and speech time.

1. Before you rent: a beginner checklist

Your $5 credit is a spending limit, not a requirement to keep a GPU running for the whole hackathon. Make local progress first. The complete handbook is the authoritative detailed procedure; this quickstart is its execution order.

Assign the four owners

P1 (MacBook) owns a small test/evidence assignment and the PPT. P2 owns the cloud, CARLA/SUMO, cameras and YOLO. P3 owns MAPPO, estimation, recovery planning and traffic evaluation. P4 owns the application/VQA/security and writes the presentation speech. Only P2 starts/stops the live simulation clock.

Run the included local starter

Install Python 3.11 and Git, unzip the kit, open `starter/` in a terminal, then run:

BASH

```
python -m unittest discover -s tests -v
python -m aegis_helpers.mock_server
```

On macOS use `python3` if needed; on Windows use `py -3.11` if installed through the launcher. Open `http://127.0.0.1:8000`. The large MOCK banner is intentional. Four fixture junctions and a camera-drop button let the team agree on data fields before any paid work. Nothing here is a trained controller or a benchmark result.

Do the account preparation locally

Create the shared repository and invite all teammates. Each person creates their own SSH key and shares only the `.pub` file with the owner. Verify model-provider and Langfuse accounts without enabling unapproved paid billing. Record the current cloud balance. Do not upload private SSH keys or API secrets to the chat or repository.

2. Choose one cloud path and check the price

Read handbook sections 2 and 5 before clicking Rent. Inspect the offer's compute, storage and transfer rates, allocated CPU/RAM, disk, host duration, and deployment type. Prices are marketplace offers, not the illustrative rates in the plan. [R01] [R03]

A full Linux VM can support normal Docker Compose workflows but may cost more in storage/boot overhead. An ordinary Vast Docker instance is already a container: run the CARLA binary inside it under a non-root user; do not assume nested Docker works. Both paths must pass the graphics test. [R02] [R04]

BASH

```
# LOCAL: example inputs only, not a quote.
python -m aegis_helpers.budget --credit 5 --compute 0.30 \
  --storage 0.02 --transfer 0.50 --reserve 1.00
```

The tool prints an estimated running-time allowance. It does not access the account or stop a bill. Keep a manual buffer for transfer and storage. Twelve/sixteen hours of continuous rental are not guaranteed affordable.

3. Connect and verify the rental

Use the exact SSH connection command provided by Vast. Add a local SSH alias only after copying the correct host, port, user and any proxy settings. Configure all necessary VM keys before creation. Read handbook section 6 for Windows/macOS commands. [R06]

BASH

```
# CLOUD: from the starter folder, read-only checks.
bash scripts/preflight.sh
```

Acceptance requires Linux x86-64, sufficient allocated resources, and Vulkan using the assigned NVIDIA GPU. `nvidia-smi` alone is insufficient. Do not install host drivers inside a rental container. [R09]

Open VS Code Remote SSH only after a plain SSH terminal works. Confirm the editor is connected to the intended host. Keep separate working copies; never let four people edit one live checkout. [R23]

4. Get one real frame before adding features

Follow handbook sections 7-8 exactly for the selected runtime. Create a non-root simulator account, a separate Python 3.10 CARLA client environment, and the official 0.9.15 server package. The Python package is not the server. Do not compile Unreal Engine during the event. [R05] [R08]

BASH

```
# CLOUD-RUNNER, from actual CARLA_ROOT:
./CarlaUE4.sh -RenderOffScreen -quality-level=Low -nosound \
  -carla-rpc-port=2000
```

Off-screen produces images without a desktop; no-rendering produces no camera data. Use the included camera smoke probe before the synchronous coordinator starts. Open the resulting PNG and inspect it. [R07]

If this fails beyond the agreed diagnostic allowance, preserve logs and pause further spending. No dashboard work can make an incompatible graphics environment satisfy PS1.

5. Get the first real closed loop

Use the official CARLA 0.9.15 SUMO example for Town05. Let SUMO manage traffic lights. The bridge command is:

BASH

```
# CLOUD-RUNNER, in CARLA_SOURCE/Co-Simulation/Sumo:
python run_synchronization.py examples/Town05.sumocfg \
  --tls-manager sumo --step-length 0.05
```

Only this coordinator may advance the live world. Inventory actual traffic-light IDs, select four connected controlled locations, and map their lanes/actions/cameras. No guessed junction IDs. [R12] [R14] [R15]

Install YOLOv10 in its own environment and test a real camera frame. Initially use fixed-time control, clearly labeled. The first milestone is camera -> detections -> observation -> reviewed phase change -> synchronized traffic -> browser evidence. Start MAPPO training separately; do not claim a pretrained traffic checkpoint exists unless you have validated one.

6. Work independently and protect the deadline

P2 emits the shared observation messages. P3 develops against the same schema in separate SUMO workers. P4 uses fixture messages until live messages are ready. P1 prepares the PPT and result templates, then inserts actual evidence. Mock and replay labels are never hidden.

By hour 12, freeze and submit the functioning build unless organizers explicitly permit longer. The brief's event header says twelve hours. Additional hours improve measurements and presentation; they should not be the first time the required system works. [EVENT]

Before judging, stop training/benchmark jobs, load a frozen checkpoint, test two resets, export evidence, and rehearse with the actual operator laptop. Keep a labeled backup recording locally.

7. End the bill safely

Export checkpoint, configuration, logs, results, slides and a version manifest. Verify the archive on two laptops. Then use the provider console to stop/destroy as appropriate and check retained volumes/other instances. Destruction is irreversible. Closing the app or SSH does not stop billing, and a stopped instance still incurs storage charges. [R01] [R02]

The kit includes setup probes and reference helpers, not a completed AEGIS implementation. Real CARLA/SUMO compatibility, learned-control quality, VQA performance, and cloud cost remain things your team must verify on its selected environment.